TP 1 :
Premier pas avec Python + OPENCV

1. Installation de OPENCV

En seance de travaux pratiques.

2. Premier programme

Essayons maintenant un programme très simple pour nous assurer que l'installation est correctement terminée.

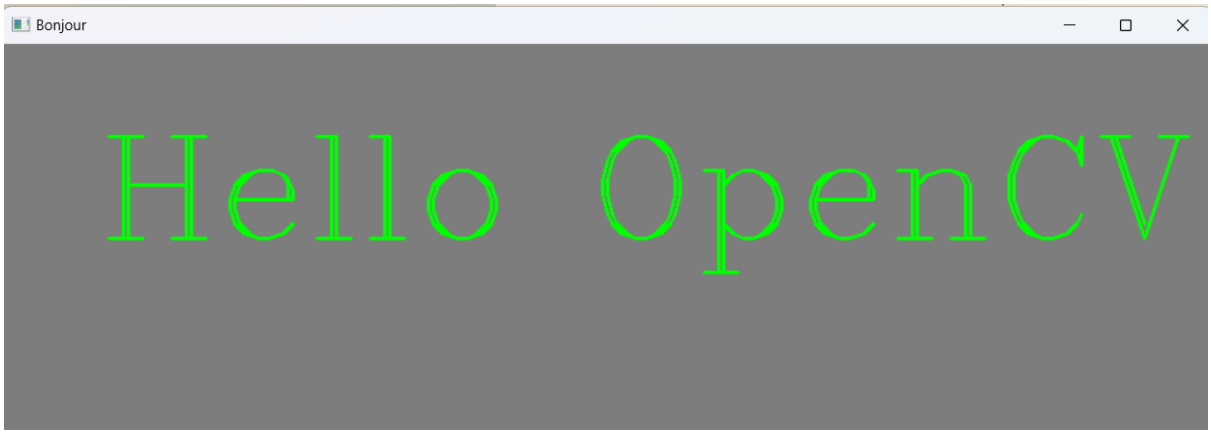
```
import cv2
import numpy as np
img = np.zeros((320,1500,3), np.uint8)#initiation de la canvas
img[:] = 125,125,125 #couleur de canvas

tx_position = (80,160)
tx_police = cv2.FONT_HERSHEY_COMPLEX #type de caractéres
tx_couleur = (0, 255, 0)#couleur de caractéres
tx_policeTaille = 4 #taille de caractéres
tx_policeEpaisseur = 2 #épaisseur de caractéres
cv2.putText(img, "Hello OpenCV", tx_position, tx_police,tx_policeTaille,
tx_couleur, tx_policeEpaisseur)
cv2.imshow("Bonjour", img)#fenetre d'affichage
cv2.waitKey(0)

cv2.destroyAllWindows()
```

Premier essai de script Python.py

Si tout est correctement installé, cette portion de code doit afficher la fenêtre suivante :



Appuyez ensuite sur n'importe quelle touche pour le fermer et terminer l'exécution. Félicitations, votre espace de travail est prêt !

3. Les basiques d'OPENCV

Cette partie de TP présentera quelques opérations de base prises en charge par OpenCV, comme l'ouverture le fichiers image ou vidéo et les afficher, convertir les images couleur en niveaux de gris, ou images noir et blanc, en se connectant à la webcam d'un ordinateur portable et montrant les vidéos capturées par celui-ci. Ce TP présentera également les

principes fondamentaux des images numériques, comme les pixels et les espaces colorimétriques.

3.1. Charger et afficher une image

Exécuter le code *Charger une image.py* ou alternativement copiez et collez les codes suivants, et faites assurez-vous d'avoir votre fichier image dans votre dossier de fichier du code et pointez-le correctement.

```
import cv2
img = cv2.imread('/resources/image1.jpg')
cv2.imshow("Image", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Explications :

Ligne 1	**import cv2** indique à Python d'inclure la bibliothèque cv2 (OpenCV). Chaque fois que vous utilisez OpenCV, vous devez importer le package cv2. C'est généralement toujours la première instruction du fichier de code.
Ligne 2	**cv2.imread()** est la fonction pour charger une image, le chemin du fichier image est précisé dans l'argument.
Ligne 3	**cv2.imshow()** est la fonction pour afficher l'image. Le premier argument est le titre de la fenêtre qui affiche l'image, le deuxième L'argument est l'image renvoyée par la fonction **cv2. imread().**
Ligne 4	**cv2.waitKey(0)** permet d'attendez que vous tapez une touche quelconque pour passer à l'instruction suivante. Sinon **cv2.imshow()** affichera la fenêtre et passera immédiatement à l'instruction la suivante, l'exécution se terminera et la fenêtre disparaîtra, cela se produit très rapidement donc on voit à peine le résultat. Donc **cv2.waitKey()** est généralement ajouté ici pour attendre qu'un utilisateur appuie sur une touche.
Ligne 5	**cv2.destroyAllWindows()** détruis toutes les fenêtres avant la fin de l'exécution.

cv2.imread() est une fonction OpenCV utilisée pour charger l'image depuis un fichier, voici sa syntaxe :

Syntaxe	***cv2.imread(chemin, indicateur)***
Paramètres :	***chemin*** : Le chemin de l'image à lire. ***indicateur*** : Spécifie comment lire l'image, la valeur par défaut est **cv2.IMREAD_COLOR**. **cv2.IMREAD_COLOR** : Charge une image couleur. La transparence de l'image sera négligée. C'est le drapeau par défaut. Vous pouvez également utiliser 1 pour ce drapeau.

	cv2.IMREAD_GRAYSCALE : Charger une image en mode niveaux de gris. Vous pouvez également utiliser 0 pour cet indicateur. **cv2.IMREAD_UNCHANGED** : Charger une image en tant que telle y compris le canal alpha. Vous pouvez utiliser -1 pour cet indicateur.
Valeur du retour	L'image chargée à partir du fichier image spécifié dans le paramètre de chemin.

</> Essayer de charger maintenant de charger et afficher l'image en niveaux de gris.

3.2. Convertir une image en niveau de gris

Une autre manière d'obtenir une image en niveaux de gris consiste à charger le fichier image originale d'abord, puis utilisez la fonction **cv2.cvtColor()** pour la convertir en image en niveaux de gris, de cette façon, nous aurons à la fois des images originales et en niveaux de gris pour un traitement ultérieur. Ceci est utile car à l'avenir, lorsque nous effectuerons le traitement de l'image, nous souhaitons traiter l'image en mode niveaux de gris tout en affichant la couleur d'origine.

```
import cv2
img = cv2.imread('ressources/image1.jpg')
gris = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
cv2.imshow("Image couleur", img)
cv2.imshow("Image niveaux de gris", gris)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

cv2.cvtColor() est une fonction OpenCV utilisée pour convertir une image d'un espace colorimétrique à un autre, voici sa syntaxe :

Syntaxe	***cv2.cvtColor(src, code[, dst[, dstCn]])***
Paramètres :	***src*** : image source à convertir. ***code*** : code de conversion de l'espace colorimétrique. ***dst*** : facultatif, l'image de sortie de la même taille et profondeur comme image src. ***dstCn*** : facultatif, le nombre de canaux dans le image de destination. Si le paramètre est 0 alors le nombre des canaux est dérivé automatiquement de l'image src. **cv2.COLOR_BGR2HSV/COLOR_HSV2BGR** : Convertir entre l'image couleur BGR et l'espace HSV. **cv2.COLOR_BGR2GRAY/COLOR_GRAY2RGB** : Convertir entre l'image couleur BGR et l'image en niveaux de gris Les autres codes de conversion ne sont pas répertoriés ici, veuillez référencer les documents OpenCV.
Valeur du retour	L'image convertie à partir de l'image source

</> Enregistrez l'image de niveaux de gris obtenues dans un nouveau fichier.

3.3. Charger et afficher des vidéos

Source : Charger video.py

Nous avons pu charger et afficher une image, nous allons maintenant travaillant avec des vidéos pour voir comment OpenCV peut traiter des vidéos.

Ouvrez le fichier ShowVideo.py ou créez un nouveau fichier Python et assurez-vous que vous disposez d'un fichier vidéo. Il existe un fichier vidéo mp4 appelé video1.mp4 dans le dossier ressources que vous pouvez utiliser.

Ci-dessous le code,

```
import cv2

capture = cv2.VideoCapture('ressources/video1.mp4')

succes, img = capture.read()
while succes:
    cv2.imshow("Video", img)
    # taper la touche ESC pour stopper la boucle
    if cv2.waitKey(10) & 0xFF == 27:
        break
    succes, img = capture.read()

capture.release()

cv2.destroyWindow("Video")
```

cv2.waitKey(10) attendra 10 millisecondes entre chaque image, la modification de la valeur du paramètre modifiera la vitesse de lecture de la vidéo. Si la valeur du paramètre est inférieure, la vitesse de lecture est plus rapide, plus grande puis plus lente.

</> À l'intérieur de la boucle while, vous pouvez traiter chaque image avant de l'afficher, par exemple vous pouvez convertir chaque image en niveaux de gris et l'afficher, vous lirez la vidéo en niveaux de gris. Cela convertira l'image de la couleur en niveaux de gris pour chaque image à l'intérieur de la vidéo, par conséquent, la vidéo sera lue en niveaux de gris.

De même, vous pouvez effectuer d'autres traitements d'image à l'intérieur de la boucle, par exemple, reconnaître les personnes ou les visages et les mettre en valeur dans le vidéo. Ceux-ci seront expliqués plus tard dans les prochains TP.

3.4. Afficher le flux à partir de la Webcam

Source : Affiche Webcam.py

Comme pour l'affichage de vidéos, une technique similaire est utilisée pour afficher une webcam. **capture = cv2.VideoCapture(0)** chargera la webcam par défaut de l'ordinateur portable/bureau et l'affichera. Dans la le code ci-dessus, le paramètre de la fonction **cv2.VideoCapture()** était le chemin du fichier vidéo, passez maintenant l'index du périphérique webcam comme paramètre, ici 0 est utilisé comme webcam par défaut, il se connectera à la webcam par défaut.

Assurez-vous qu'une webcam est connectée à l'ordinateur portable/bureau et activée, exécutez le code, vous verrez une fenêtre affichant tout ce que la webcam capture. Appuyez sur la touche ESC pour terminer.

```
import cv2

capture = cv2.VideoCapture(0)

capture.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
capture.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
capture.set(cv2.CAP_PROP_BRIGHTNESS, 180)
capture.set(cv2.CAP_PROP_CONTRAST, 50)

succes, img = capture.read()
while succes:
    cv2.imshow("Webcam", img)

    if cv2.waitKey(10) & 0xFF == 27:
        break
    succes, img = capture.read()

capture.release()

cv2.destroyWindow("Webcam")
```

Pour références, voici la liste qui peut être utilisée comme paramètre pour *capture.set()* :

https://docs.opencv.org/3.4/d4/d15/group__videoio__flags__base.html#gaeb8dd9c89c10a5c63c139bf7c4f5704d